UWE Bristol | University of the West of England

Lecture 3: Object Oriented Modelling & Analysis

Week 3

Outline

- *Modelling and requirement engineering review*
- *Analysis: Use Case Diagrams*
  - *Actors*
  - *Use Cases*
  - *Relationships*
- *Case Study*

Modelling

Modelling consists of building an abstraction of reality

Abstractions are simplifications because:

They ignore irrelevant details and

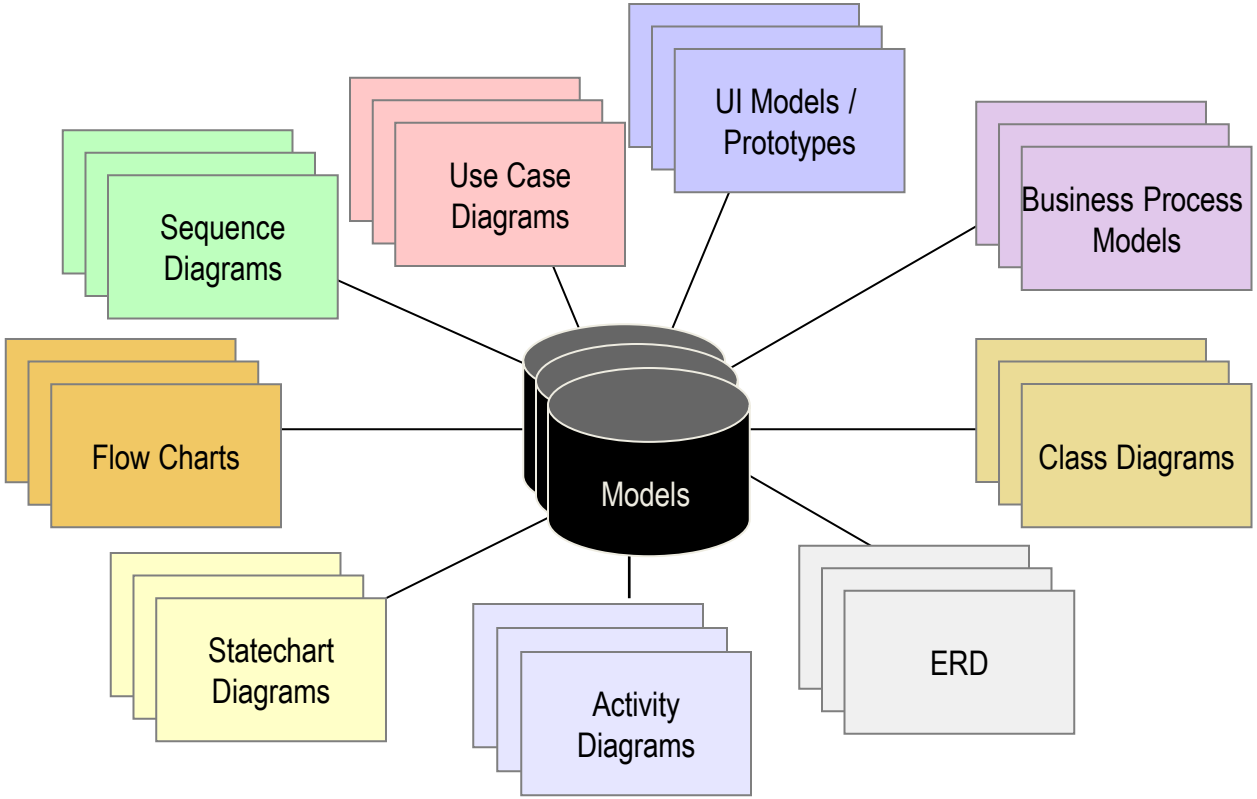
They only represent the relevant details

What is *relevant* or *irrelevant* depends on the purpose of the model

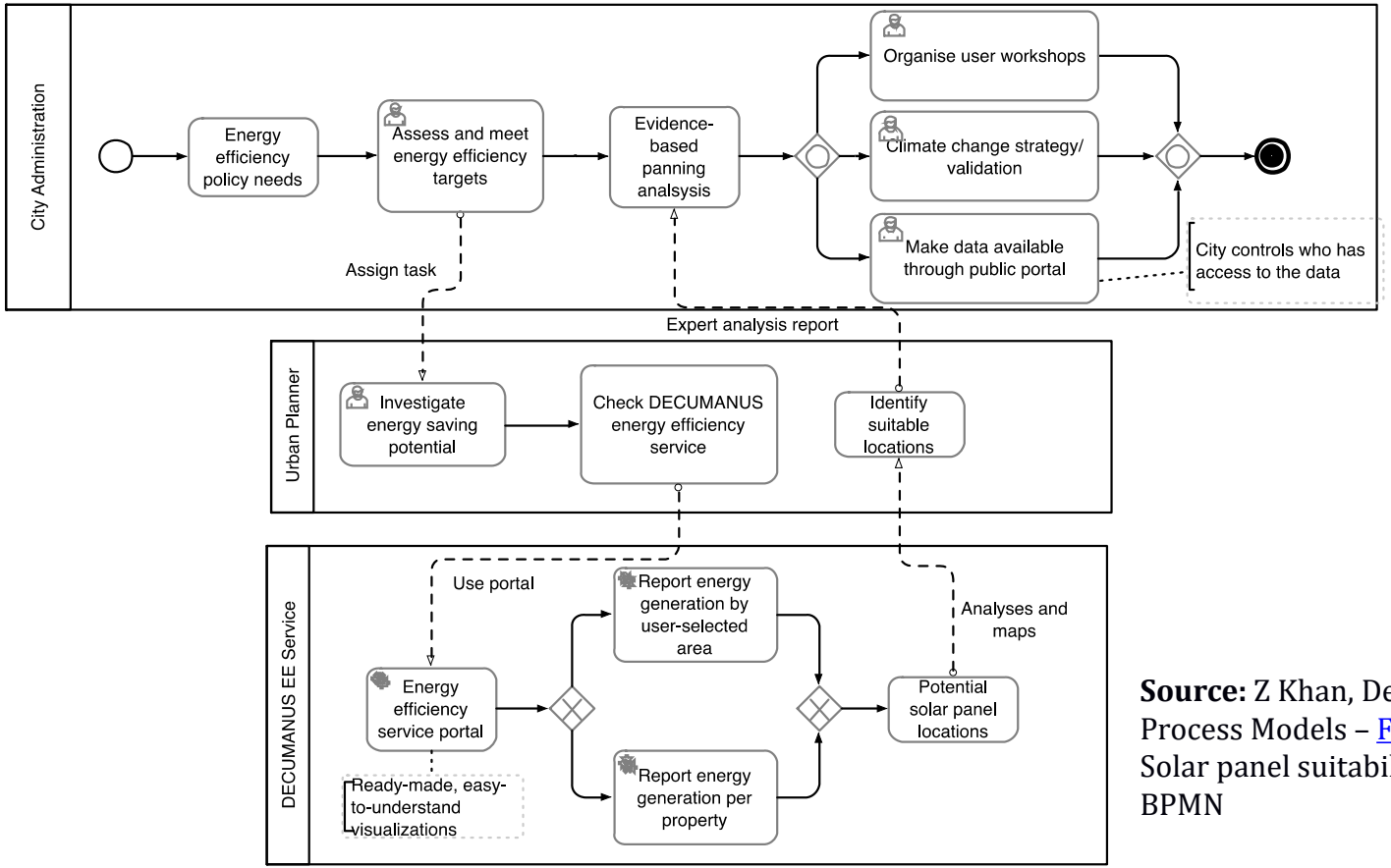
We use the term ***system*** as a ***collection of interconnected parts***.

Modeling is a way to deal with complexity by ignoring irrelevant details. We use the term ***model*** to refer to ***any abstraction of the system***.

Various Types of Model



Various Types of Model



Source: Z Khan, Deliverable D3.2 Service Usage Process Models – FP7 DECUMANUS project: Solar panel suitability scenario BPMN

Software Engineering: A Problem Solving Activity

- .....is more than just writing Code
- **Analysis**: Understand the nature of the problem and break the problem into pieces
- **Synthesis**: Put the pieces together into a large structure
- **For problem solving**: we use techniques, methodologies and tools.

Object-oriented modelling

- **Desirable properties:**
  - **Split** the overall task and assign to subsystems
  - Subsystems know only **what** task a particular subsystem does, but not how it performs the assigned task (**what,** not how)
  - Each subsystem works independently but **communicate** with others whenever necessary
  - They can be reused

Models to describe Software Systems

UWE Bristol University of the West of England

- **Object model**: What is the structure of the system? (static, specifies to whom it happens to)
- **Functional model**: What are the functions of the system? (what the application does and not how it does, specifies what happens)
- **Dynamic model**: How does the system react to external events? (essential behaviour of the application, specifies when it happens)
- **System Model**: Object model + functional model + dynamic model

Object Oriented Analysis- Process

UWE Bristol University of the West of England

- Idea
- Project Description
- Risk Analysis
- Boundary Analysis
- Actor Identification
- Use Case Identification

Conception: Idea + Project Description

UWE Bristol University of the West of England

Every project starts with an idea

- This idea should be documented and discussed

The project description describes how the idea is turned into a product

- Description should be brief
- No implementation details
- Approximate length 1 paragraph – 2 pages
- Description should be complete!
- Small group of authors

Risk Analysis

Find all relevant **market** factors, e.g.:

- Competition
- Technologies used
- Market trends
- Future trends (e.g. new mobile phones, Galileo system)

Initial **risk** analysis: what can go wrong?

- People
- System
- Resources
- Technology
- Corporate/marketing

System Identification

- **Development of a system is not just done by taking a snapshot of a scene (domain)**

- **Definition of the system boundary**
  - What is inside, what is outside?

- **How can we identify the purpose of a system?**

- **Requirements Process**
  - **Requirements Engineering**: Definition of the system in terms understood by the customer
  - **Requirements Analysis**: Technical specification of the system in terms understood by the developer

Requirements Engineering

- **Very challenging activity**
  - Elicitation & Analysis, Specification, Validation, Management

- Requirements Engineering: Processes and Techniques by Gerald Kotonya, Ian Sommerville (1998), Wiley

Requirements Engineering

UWE Bristol University of the West of England

- **Requires collaboration of people with different backgrounds**
  - User with application domain knowledge
  - Developer with implementation domain knowledge
- **Bridging the gap between user and developer:**
  - Scenarios: Example of the use of the system in terms of a series of interactions between the user and the system
  - Use cases: Abstraction that describes a class of scenarios

- **CoReS method**: Collaborative Requirements Engineering and Stakeholder engagement' Khan, Zaheer, et al. "Applying the CoReS requirements development method for building IT tools for urban management systems: the UrbanAPI project." Theoretical and Empirical Researches in Urban Management, vol. 8, no. 4, Nov. 2013, pp. 25+

Requirements Engineering

UWE Bristol University of the West of England

Requirement #21

Edit | Log time | Watch | Copy | Delete

Use Case #20: Co-creation between a developer and the community

« Previous | 6 of 69 | Next »

System should allow users to create 3D models of proposed development plans

Added by Kamran Soomro 11 months ago. Updated about 1 month ago.

Status:	Validated	**% Done:**	0%
Priority:	High		
Owner:	Hamburg, Hamburg Smartathon, RBKC, Rome	**Assumption:**	The 3D models are BIM-compliant
Source:			
Type:	Functional	**Validated:**	Hamburg, Rome
		Means of Validation:	
Rationale:	Facilitate the communication of development proposals between different stakeholders.	**Acceptance Criteria:**	Users can place various objects in a 3D model and receive feedback
Sprint:			

Description

Quote

The system should allow the users to develop 3D models of proposed developments plans by placing objects within existing 3D models. The 3D models should include basement developments.

For Hamburg the system should provide the option to add different types of trees. *See Use Case #39.*

For Rome the platform should support urban gardens.

Khan, Z.; Dambruch, J.; Peters-Anders, J.; Sackl, A.; Strasser, A.; Fröhlich, P.; Templer, S.; Soomro, K. Developing Knowledge-Based Citizen Participation Platform to Support Smart City Decision Making: The Smarticipate Case Study. *Information* **2017**, *8*, 47. https://doi.org/10.3390/info8020047

Object Oriented Analysis and the Role of ***Domain Experts***

- During the early stages of any project development, much of your time will be spent in the analysis phase, attempting to understand '*what*' the system must do and how it interacts with the real world.

- That is, what interfaces, functionality and behaviour the system should provide for your customer.

- Such analysis is often conducted in the presence of one or more '*domain experts*', i.e. somebody who is intimately familiar with the business process or procedures that you are trying to automate (i.e. the problem domain) and can describe them to somebody else, often in a non-technical way.

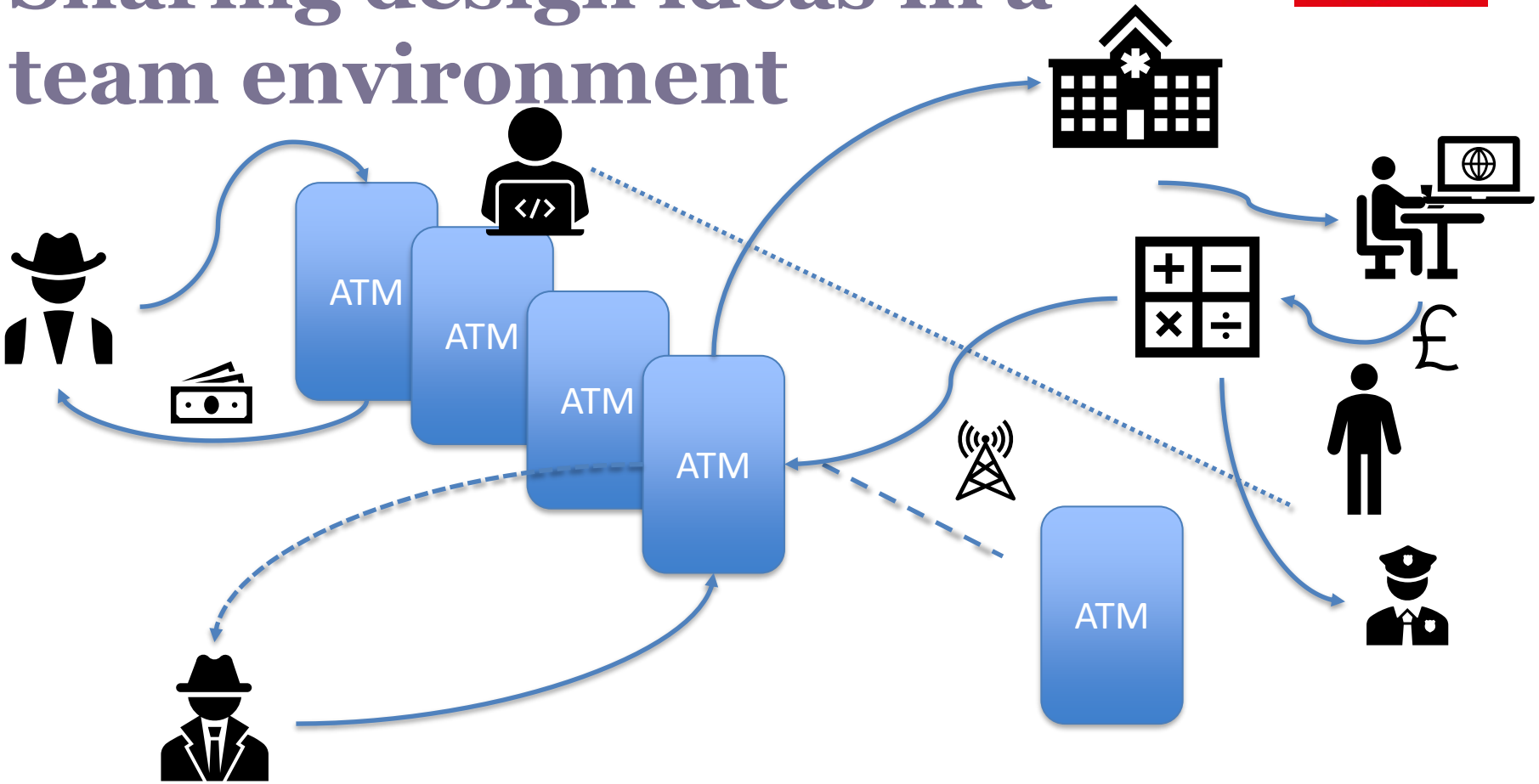
In simple terms, a domain expert is somebody who understands or knows how to do the job before it has been automated.

Example Domain Experts

A domain expert is someone who has specialised knowledge or abilities in a specific domain. An accountant, for example, is an expert in the domain of accounting.

The development of accounting software requires knowledge in two different domains, namely accounting and software.
Some of the development workers may be experts in one domain and not the other.

Sharing design ideas in a team environment



Sharing design ideas in a team environment

- If the software is large scale, employing perhaps dozens of developers over several years, it is important that all members of the development team communicate using a common language.

- That is, the ideas in the head of say the analyst have to be conveyed to the designer in some way so that he/she can implement that idea in code.

- That notation is called UML. The Unified Modelling Language. Some might prefer the title Universal Modelling language since it can be used to model many things besides software.

What is UML ?

- UML is not a language in the same way that we view programming languages such as Python, 'C++', 'Java' or 'Basic'.

- UML is however a language in the sense that it has syntax and semantics which convey meaning, understanding and constraints (i.e. what is right and wrong and the limitations of those decisions) to the reader and thereby allows two people fluent in that language to communicate and understand the intention of the other.

- UML represents a collection of 13 essentially graphical (i.e. drawing) notations supplemented by textual descriptions designed to capture requirements and design alternatives. You don't have to use them all, you just chose the ones that capture important information about the system you are working on.

What is UML ? Cont.

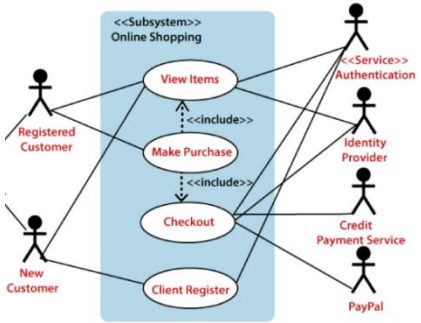
UWE Bristol University of the West of England

- **UML** (Unified Modeling Language)
  - Nonproprietary standard for modeling software systems, OMG
  - Convergence of notations used in object-oriented methods
    - OMT (James Rumbaugh and colleagues)
    - Booch (Grady Booch)
    - OOSE (Ivar Jacobson)
- Current Version: UML 2.5.1, December 2017
  - Information at the OMG portal https://www.omg.org/spec/UML/2.5.1/About-UML
  - UML1.4 and UML2.0 Naming differences: https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=oum-naming-differences-between-uml-14-later-uml-versions
- Commercial tools: Rational (IBM),Together (Borland), Visual Architect (business processes, BCD)
- Open Source tools: ArgoUML, StarUML, Umbrello
- Commercial and Opensource: PoseidonUML (Gentleware)

We are using Astah

The six most important diagram types

- Use Case Diagrams:
  - A simple but very effective model used during the analysis phase for analysing requirements through the process of exploring user interactions with the system.



  - The process involves documenting
    - Who initiates an interaction,
    - What information goes into the system,
    - What information comes out and
    - What the measurable benefit is to the user who initiates the interaction (i.e. what they get out of it).

Requirements analysis attempts to uncover and document the services the system provides to the user.

Image source: https://www.javatpoint.com/uml-use-case-diagram

The six most important diagram types Cont.

- Class Diagrams:
  - A powerful tool for exploring architecture, functionality and relationships between objects in our system (i.e. instances of classes).

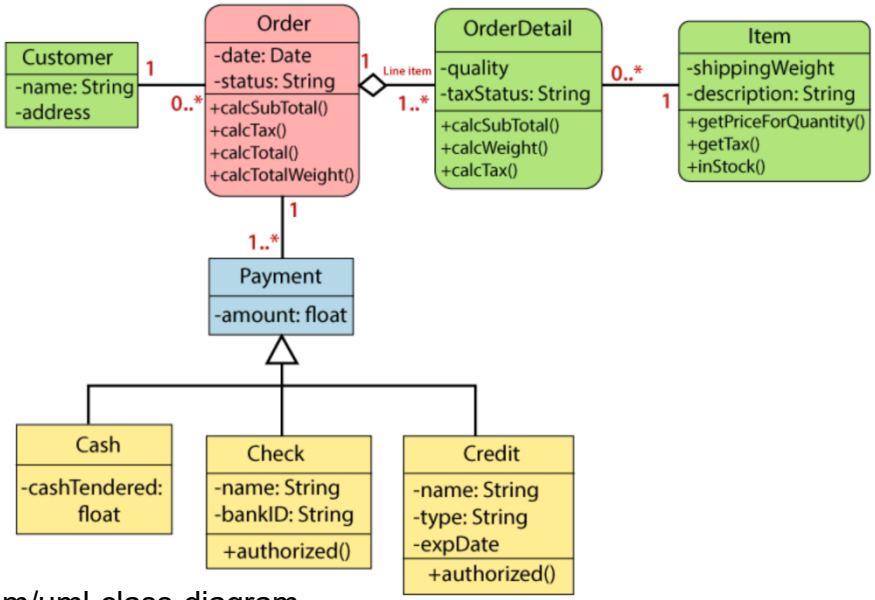
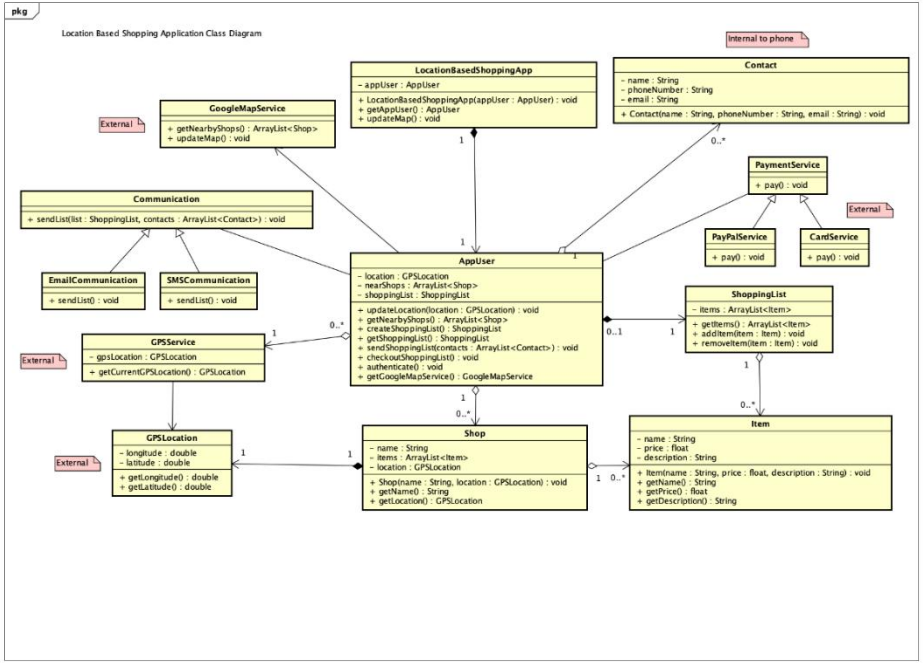


Image source: https://www.javatpoint.com/uml-class-diagram

The six most important diagram types Cont.

- Class Diagrams:
  - A powerful tool for exploring architecture, functionality and relationships between objects in our system (i.e. instances of classes).



The six most important diagram types Cont.

- Sequence and Communication (*Collaboration*) Diagrams:
  - These two diagrams model the interaction of a set of collaborating objects through a process of message passing as they attempt to achieve the functionality expressed in one or more use cases.
  - In essence they model the behaviour of our system in response to inputs from the external world.

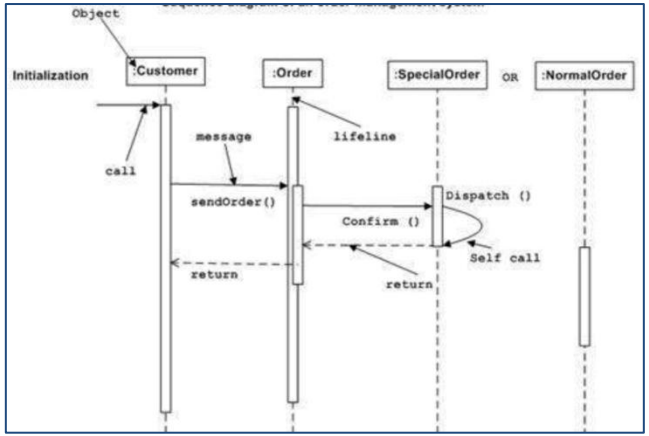


Image source:
https://www.tutorialspoint.com/uml/uml_interaction_diagram.htm

The six most important diagram types Cont.

- State Charts:
  - An extension of state transition diagrams.
  - Model the time dependent behaviour of objects or systems in response to messages sent to it over a period of time.

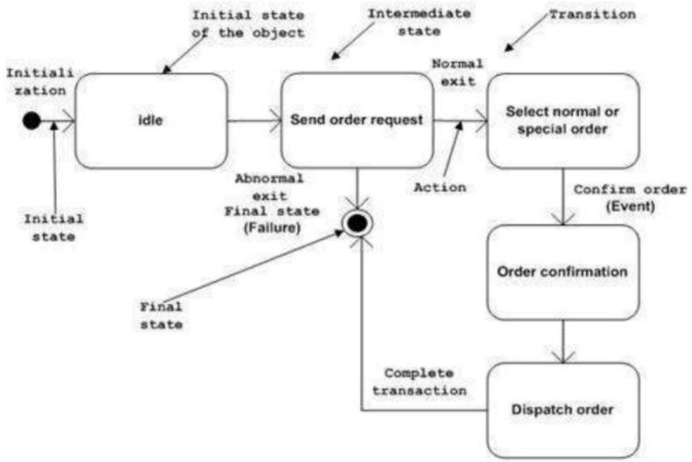


Image source:
https://www.tutorialspoint.com/uml/uml_statechart_diagram.htm

The six most important diagram types Cont.

- Activity Diagrams.
  - Used during analysis to explore areas of parallelism or concurrency in the customer business model.
  - Useful in areas of business systems modelling where many processes or activities within the business are carried out in parallel, e.g. simultaneously raising an invoice while at the same time producing a delivery note and shipping the goods or order.
  - Gives an indication to the designer of things that happen in parallel and things that happen in sequence.
  - A bit like a flowchart but with parallelism and synchronisation built in.
  - Can be useful in modelling concurrent processes/threads
- See example here: https://www.researchgate.net/publication/228828850_Modeling_processes_using_RAD_and_UML_activity_diagrams_An_exploratory_study

The six most important diagram types Cont.

- We can model 80% of most problems by using about 20% UML

- we will consider those 20%

User Interaction Analysis – Use Cases

A formal way of representing how a business system interacts with its environment

Illustrates the activities that are performed by the users of the system

A scenario-based technique in the UML

A sequence of actions a system performs that yields a valuable result for a particular actor.

User Interaction Analysis –Cont.

Use Cases

- What is an Actor?
  - A user or outside system that interacts with the system being designed in order to obtain some value from that interaction

- Use Cases describe scenarios that describe the interaction between users of the system (the actor) and the system itself.

User Interaction Analysis –Cont.

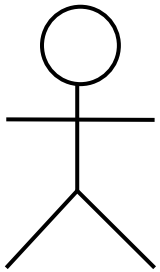
Use Cases

- **Use case diagrams** describe what a system does from the standpoint of an external observer. The emphasis is on *what* a system does rather than *how.*

- Use case diagrams are closely connected to scenarios. A **scenario** is an example of what happens when someone interacts with the system.

Use Case Diagram (Actor)

- An **Actor** is a model for an external entity which interacts (communicates) with the system/plays a role in one or more interactions with your system
  - **User**
  - **External system** (Another system)
  - **Physical environment** (e.g. Weather)
- An actor has a unique name and an optional description
- Examples:
  - **Customer**: A person in the bank
  - **GPS satellite**: An external system that provides the system with GPS coordinates.



Customer

Name

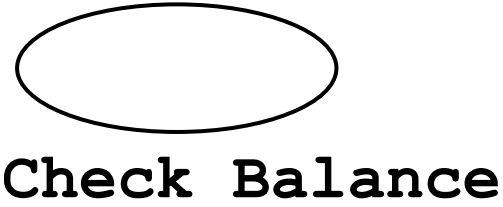
Optional Description

Actors Vs. Stakeholders

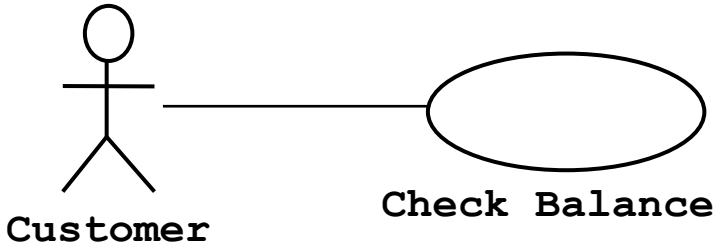
- Stakeholder: anyone interested in the system
  - examples: supplier, stock agency, vendor
  - stakeholder might not "act" in any scenario
    - someone might be investing but will never use the system

Use Case Diagram (Use Case)

- A **Use Case** is a set of scenarios tied together by a common user goal
- A **scenario** is a sequence of steps describing an interaction between a user and a system
- Use cases can be described textually, with a focus on the event flow between actor and system
- The textual use case description consists of 6 parts:
    1. Unique name
    2. Participating actors
    3. Entry conditions
    4. Exit conditions
    5. Flow of events
    6. Special requirements.



Graphical as well as Textual Use Case Example

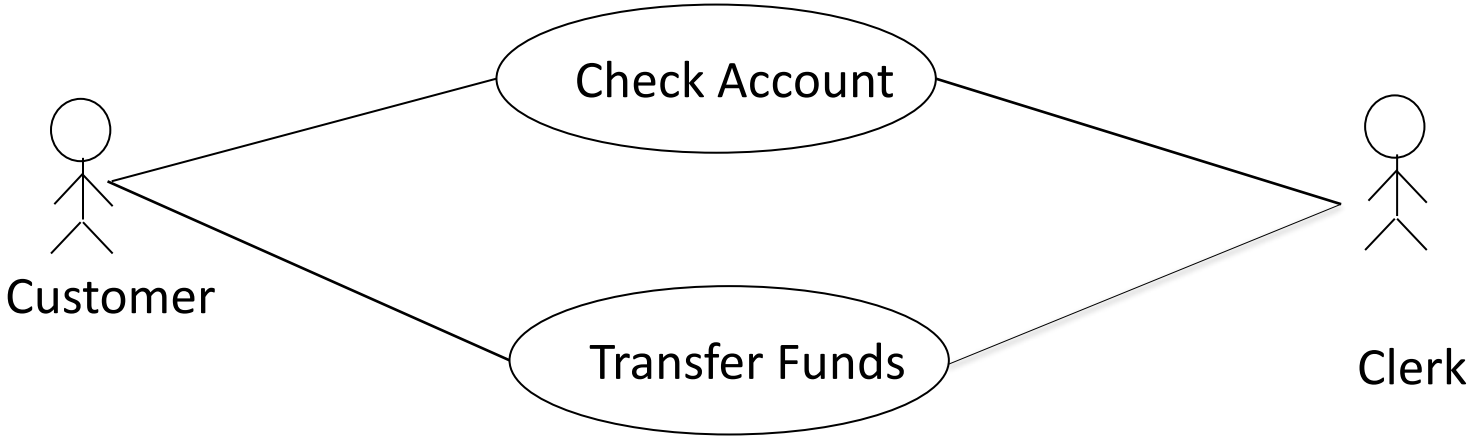


Noun-verb naming convention
Use case: Verb phrase

1. *Name:* Check Balance
2. *Participating actor:* Customer
3. *Entry condition:*
   - Customer in front of computer/smartphone
4. *Exit condition:*
   - Customer checked his account balance
5. *Flow of events:*
   1. Customer logs in to the system
   2. System displays the account number(s)
   3. Customer selects an account
   4. System displays account balance
6. *Special requirements:* *None.*

What happens: if the customer cannot login?

Association Between Actors and Use Cases



- An **actor** **must be** associated with **at least one** use case;
- An **actor** **can be** associated with **multiple** use cases;
- **Multiple actors** **can be** associated **with a single** use case.

Use cases vs. internal features

- Create Account
- Check Balance
- Deposit Funds
- Withdraw Cash
- Transfer Funds

Point of view: user

- initializes balance, owner, and account number
- user I/O (display)

.

.

.

Point of view: developer

Characteristics of a good use case

UWE Bristol University of the West of England

- starts with a request from an actor to the system
- ends with the production of all the answers to the request
- defines the interactions (between system and actors) related to the function
- takes into account the **actor's point of view**, not the system's
- focuses on interaction, not internal system activities
- doesn't describe the GUI in detail
- has 3-9 steps in the main ***success*** scenario
- is easy to read

Use Case Diagrams

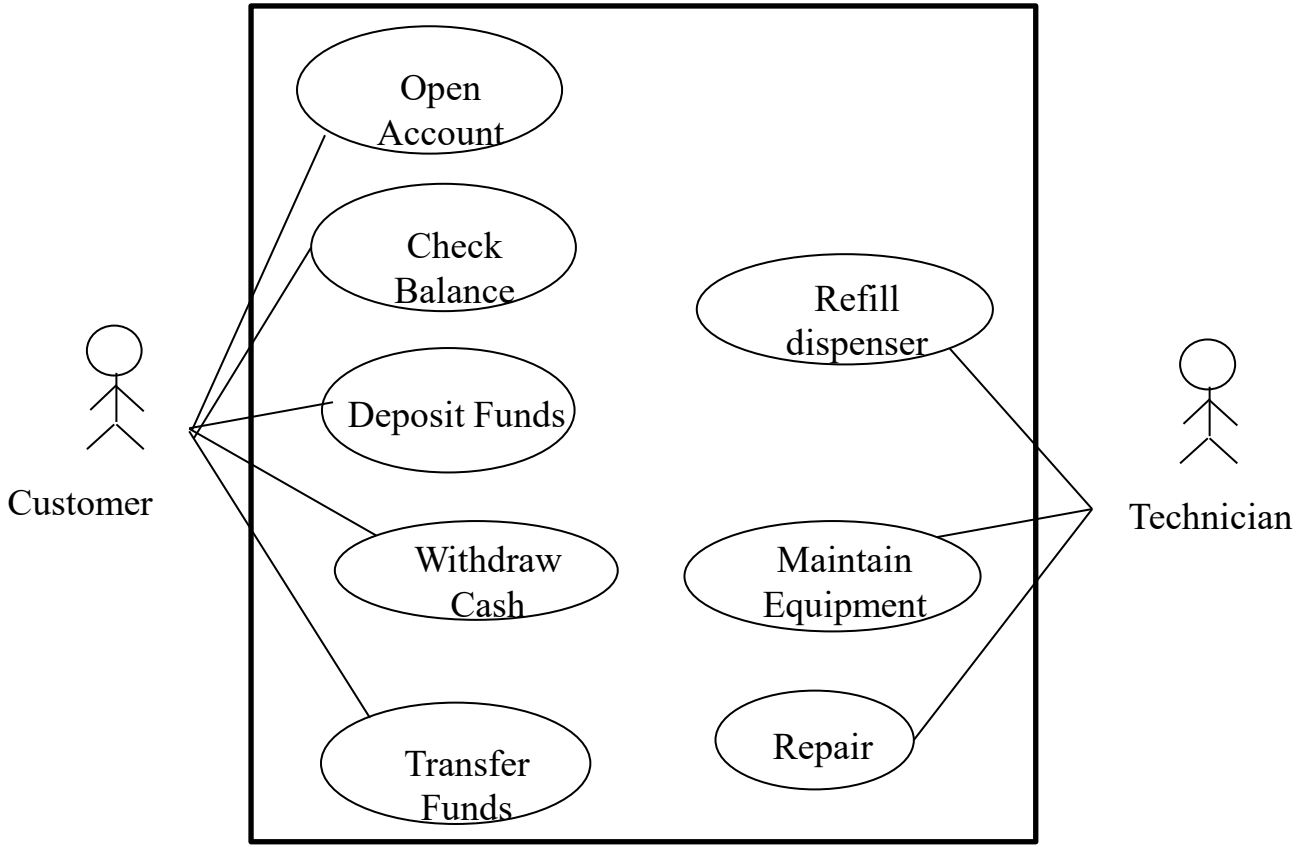
- **A use case diagram is a graphical view of some or all of the actors, use cases, and their interactions identified for a system**
- Each system typically has a Main Use Case diagram
  - A picture of the system boundary (actors) and the major functionality provided by the system (use case packages)
- Other use case diagrams may be created as needed
  - A diagram showing all the use cases for a selected actor
  - A diagram showing all the use cases being implemented in an iteration
  - A diagram showing a use case and all of its relationships

How to draw a Use Case Diagram?

- We need to identify:
  - Possible **Actors**
  - Functionalities to be represented as an **Use Case**
  - **Relationships** among the **Use Cases** and **Actors**

- After identifying the above items, following guidelines have to be followed to draw an efficient Use Case diagram:
  - Suitable name should be chosen so that it can identify the functionalities performed
  - Choose suitable names for Actors
  - Show relationships and dependencies clearly in the diagram
  - Provide relationships those are necessary
  - Use note whenever required to clarify some important points

An example diagram



Relationships between Use Cases

- **Generalization** - use cases that are specialised versions of other use cases

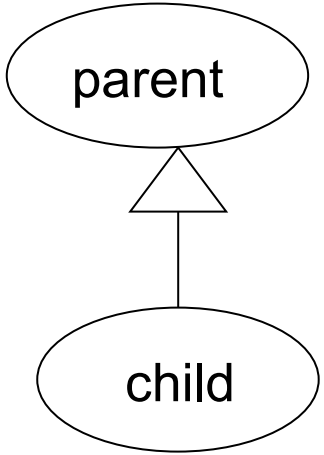
- **Include** - use cases that are included as parts of other use cases. Enable to factor common behavior e.g., login before checking balance

- **Extend** - use cases that extend the behavior of other core use cases. Enable to factor variants. E.g., print receipt after checking balance

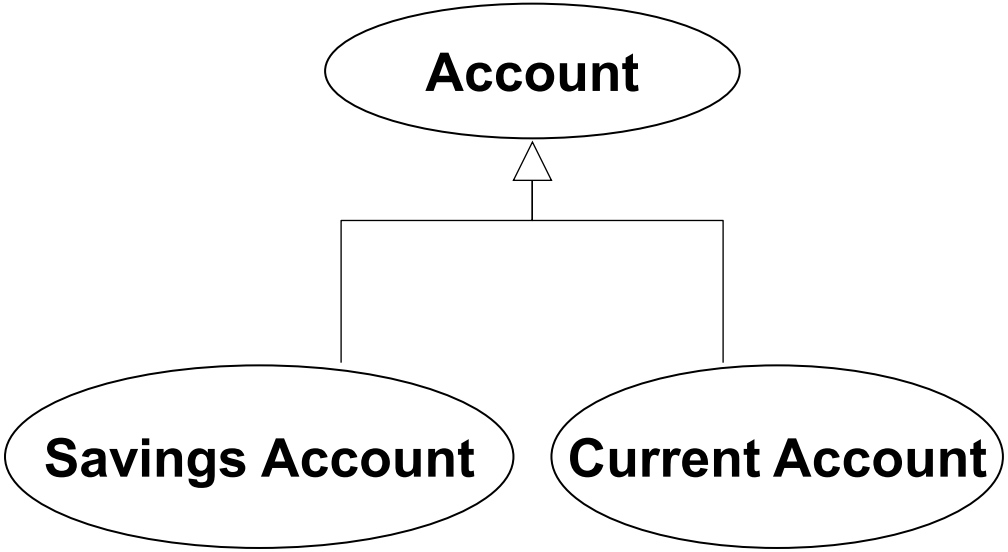
Generalization

- The child use case inherits the behavior and meaning of the parent use case

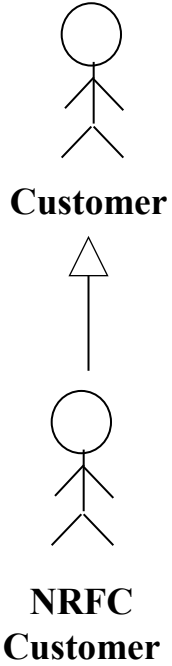
- The child may add to or override the behavior of its parent



Generalization cont.

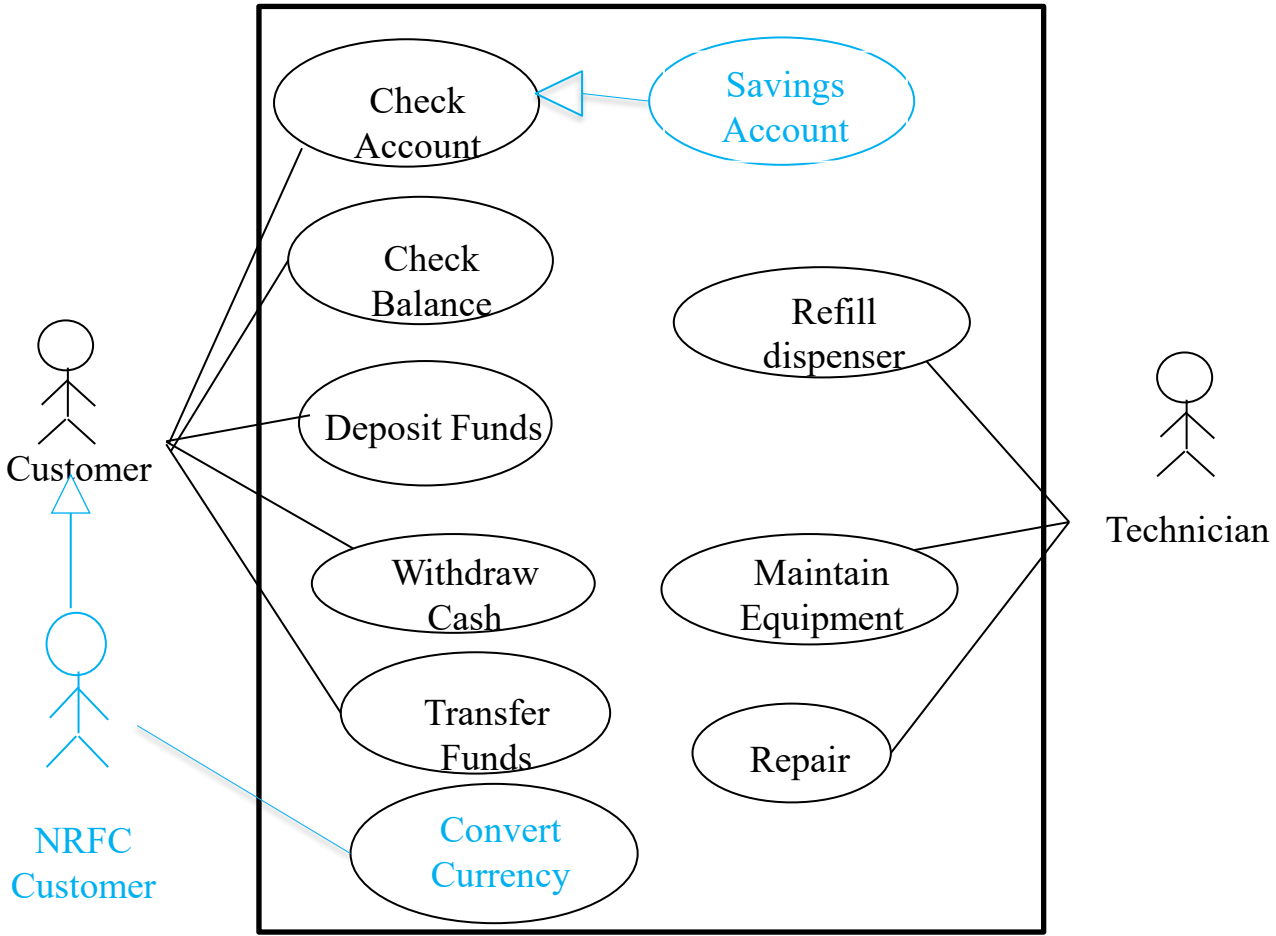


Use Case Generalization

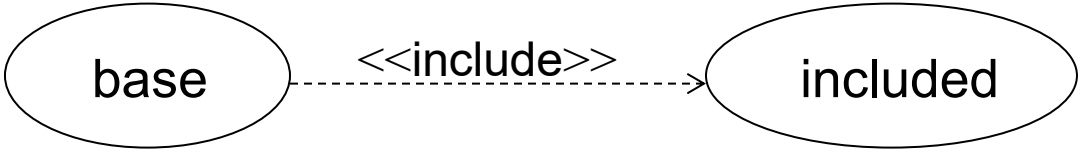


Actor Generalization

Example diagram cont.(Generalization)

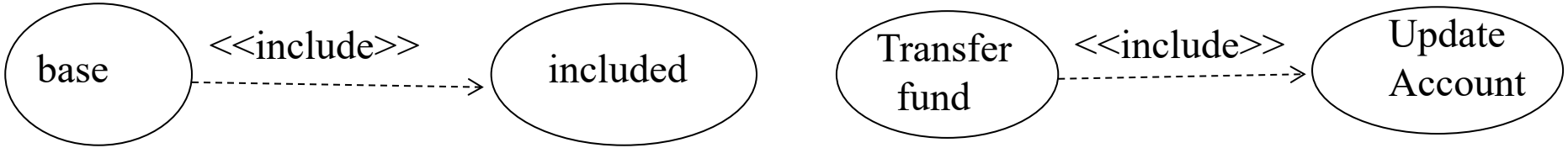


Include relationship



- The **base use case explicitly incorporates the behavior of another use case** at a location specified in the base
- The included use case never stands alone. It only occurs as a part of some larger base that includes it
- I.e., <<**include**>> relationship represents common functionality needed in more than one use case

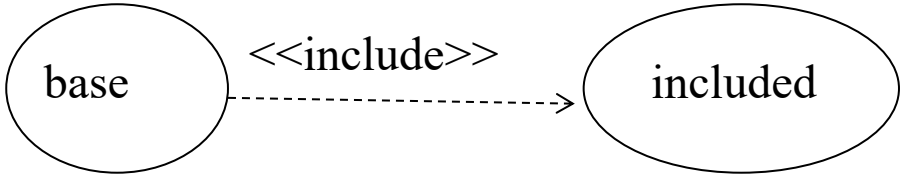
Include relationship cont.

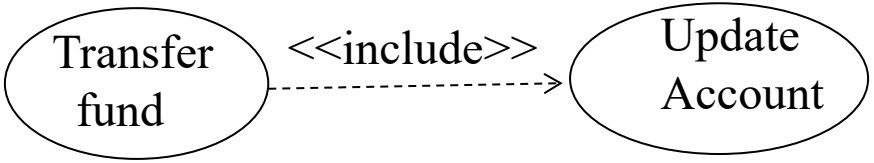


Included use case
Is this (included) use case optional?
Is the base use case complete without this (included) use case?
Is the execution of this (included) use case conditional?
Does this (included) use case change the behavior of the base use case?

Table source © Ivan Marsic

Include relationship cont.

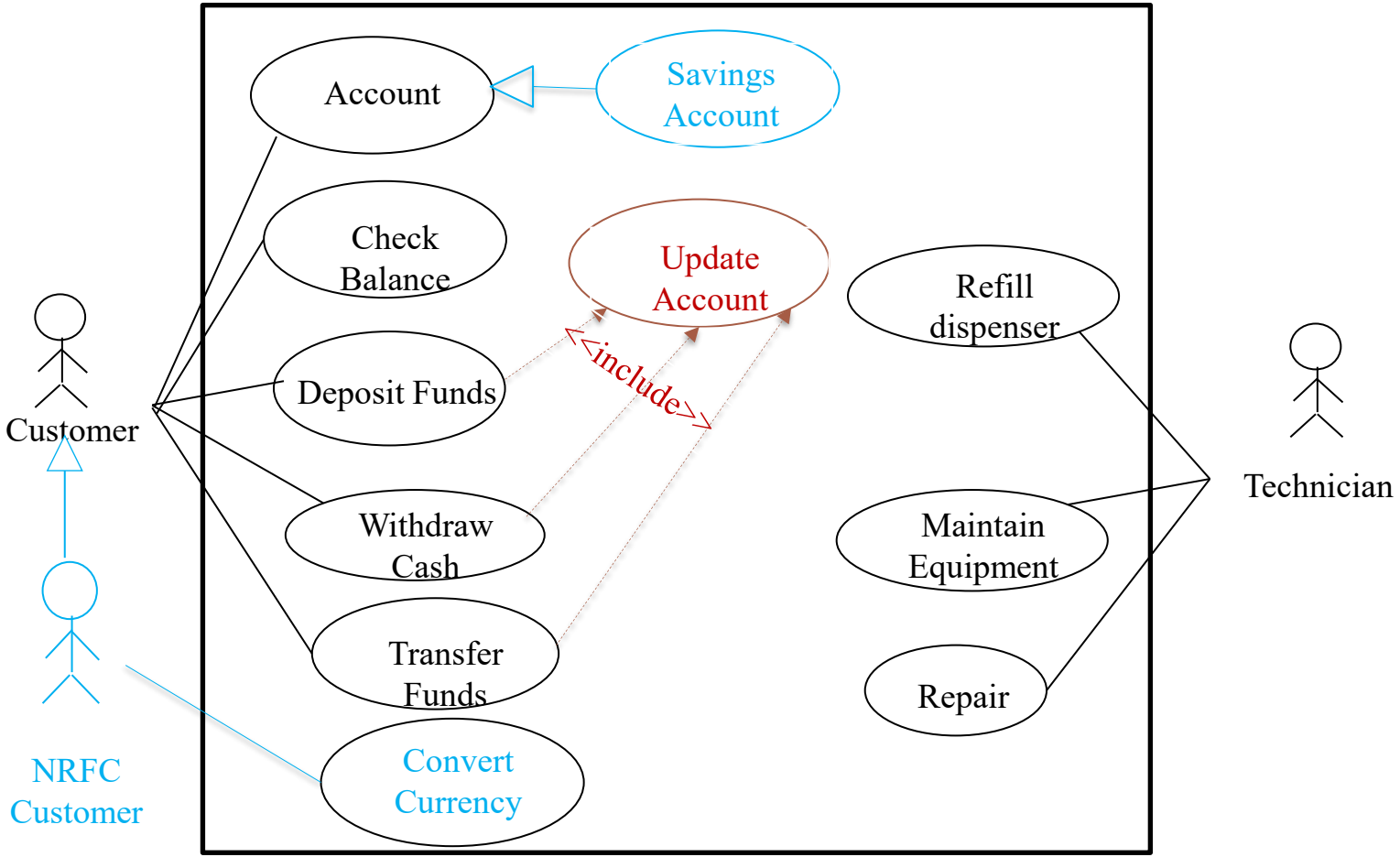




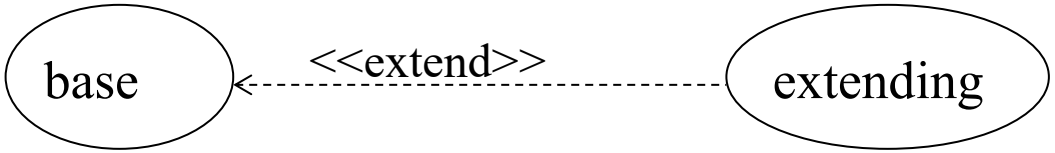
Included use case	
Is this (included) use case optional?	NO
Is the base use case complete without this (included) use case?	NO
Is the execution of this (included) use case conditional?	NO
Does this (included) use case change the behavior of the base use case?	NO

Table source © Ivan Marsic

Example diagram cont.(include)



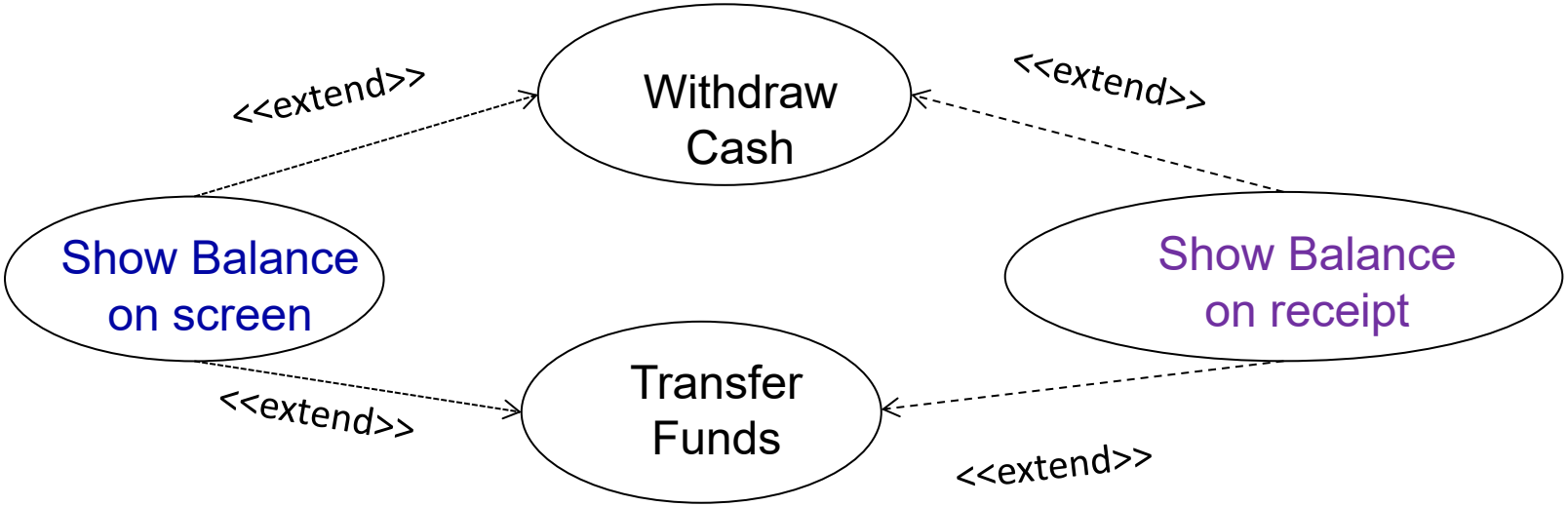
Extend relationship



- The base use case implicitly incorporates the behavior of another use case at certain points called extension points
- The base use case may stand alone, but under certain conditions its behavior may be extended by the behavior of another use case

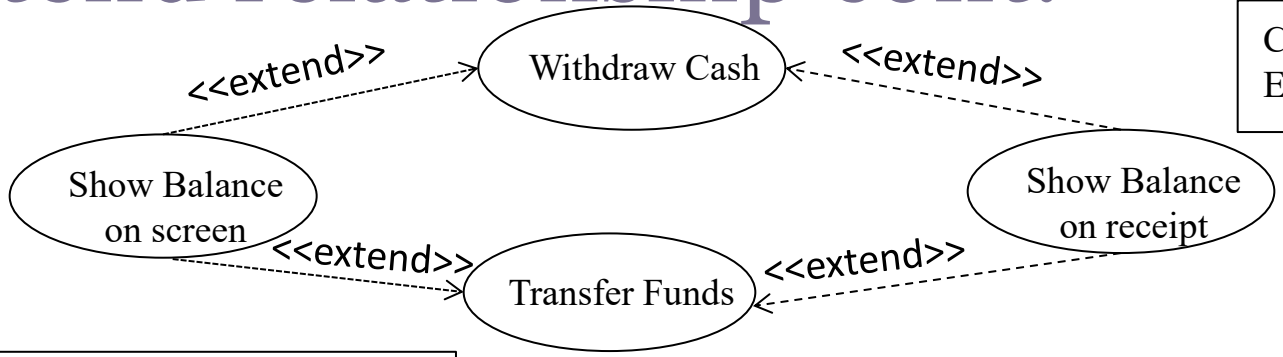
Extend relationship cont.

Condition: receipt output
Extension: balance information



Condition: screen output
Extension: balance information

Extend relationship cont.



Condition: receipt output
Extension: balance information

Condition: screen output
Extension: balance information

Extending use case

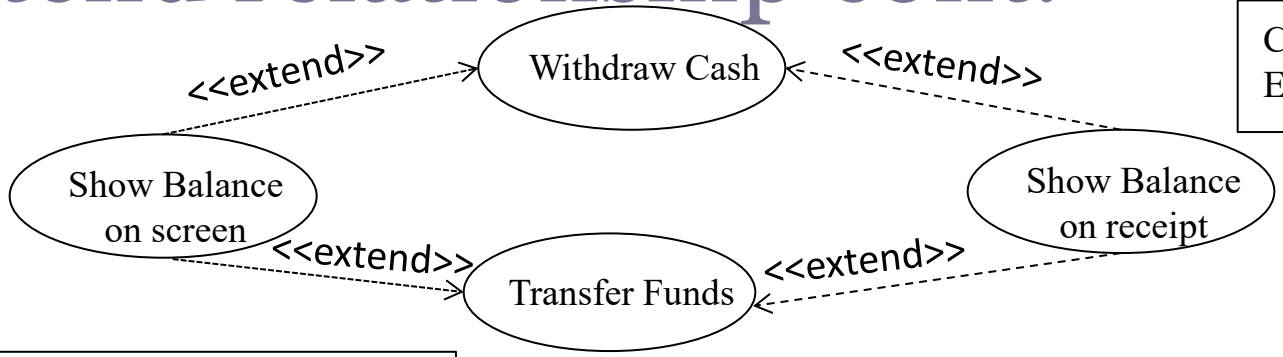
Is this use case (extended) optional?

Is the base use case complete without this use case (extended) ?

Is the execution of this use case (extended) conditional?

Does this use case (extended) change the behavior of the base use case?

Extend relationship cont.

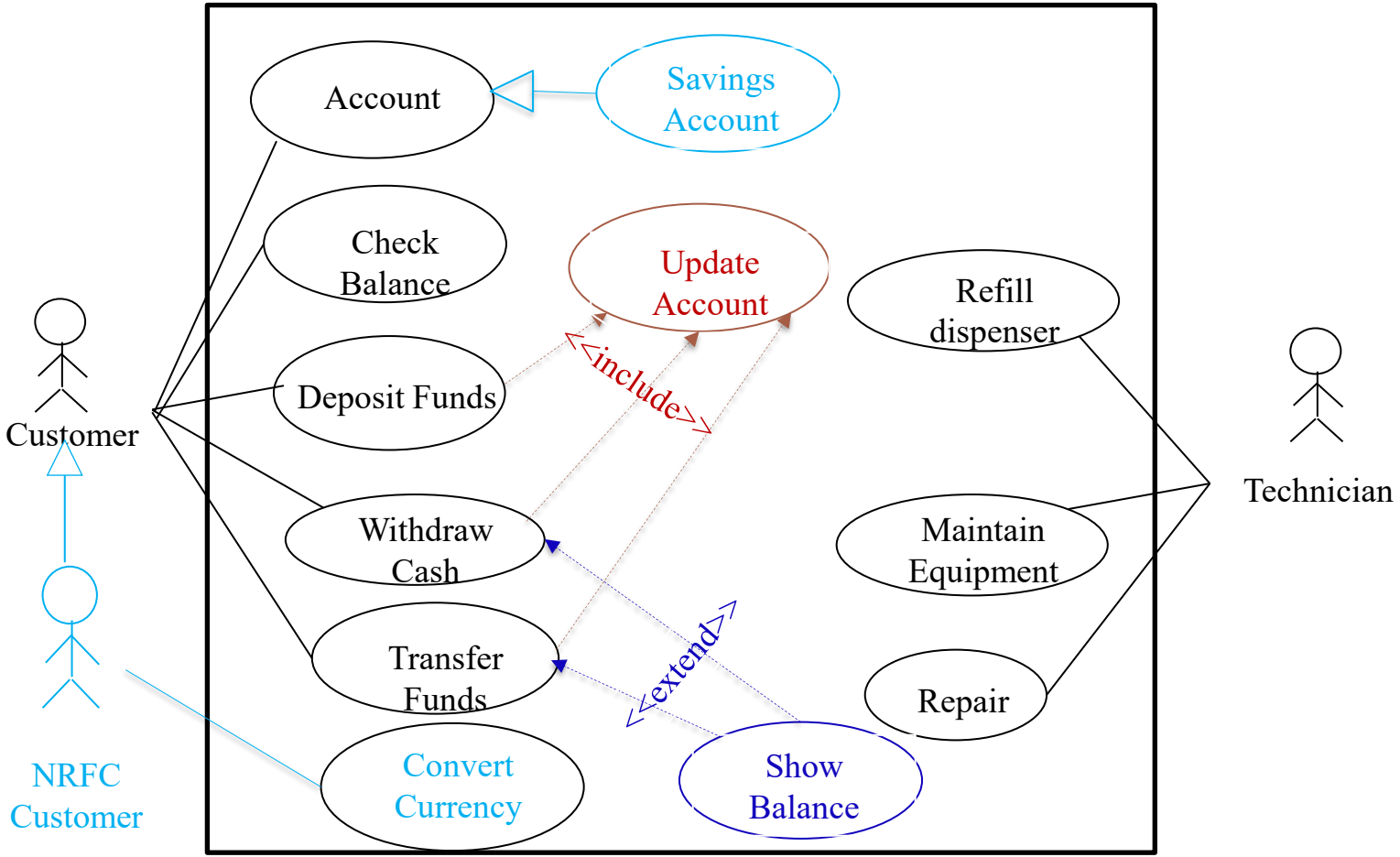


Condition: receipt output
Extension: balance information

Condition: screen output
Extension: balance information

Extending use case	
Is this use case (extended) optional?	YES
Is the base use case complete without this use case (extended) ?	YES
Is the execution of this use case (extended) conditional?	YES
Does this use case (extended) change the behavior of the base use case?	YES

Example diagram cont.(extend)



Case Study

Object Oriented Analysis

UWE Bristol University of the West of England

- Idea
- Project Description
- Risk Analysis
- Boundary Analysis
- Actor Identification
- Use Case Identification

Make a project description

UWE Bristol University of the West of England

The ‘Just in time 4 my bus’, or JIT-bus mobile phone app allows a user to select a bus line from nearby bus stops on a map, and will set an alarm notifying the user when she has to leave to catch the next possible bus of the selected line.

Group task: Write the corresponding project description!

‘Just in time 4 my bus’- Project description

- We are developing a mobile phone app that let’s people leave just in time to catch their bus: no earlier and no later.
- The app lets the user select a bus line they want to use from a number of nearby bus stops.
- The app sounds an alarm when the user needs to leave.
- The app incorporates live bus location times to make sure the alarm is sounded at the right moment, even if a bus is too late or too fast

Risk analysis of JIT-bus

- Make a list of market factors for the JIT-bus app

- Do a risk analysis for the app

Risk analysis of JIT-bus cont.

UWE Bristol University of the West of England

Market factors:

- There are already a number of bus time apps available in the market, some of them have the option to pick a line from a map. None have an alarm though.
- The system depends on the accuracy of bus localisation provided by the bus companies.
- The system needs users with GPS-enabled smart-phones.

Risk factors:

- No experience in writing mobile phone apps.
- Individual differences in time needed to get to bus stop.
- GPS can’t tell how long it takes to get out of building.
- GPS can’t tell what floor we’re on.
- Bus times may be very inaccurate.
- Difficult to advertise apps – no budget.

System Boundary

- System boundary defines the separation between the system and its environment
- Clear definition is extremely important
- Movement of the system boundary often has a large effect on what should be built
- A common area of ***conflict*** between ***stakeholders*** arises when they assume different system boundaries, and hence refer to different system

Boundary Analysis- JIT Bus

OUT

GPS system

Bus times

IN

Bus stop map
List of bus lines for each bus stop
`Leave now!' alarm
Show current location

Map databases

Alarm sounds

Out of scope:
Route planner

Identifying Actors

- Actors are anyone or anything that interacts with your system
- Actors are always outside of your system

Identifying actors:

- Who uses the system?
- Who installs the system?
- Who starts/stops the system?
- Who maintains the system?
- Who gets information from the system?
- What other systems use the system?

Identify Actors cont.

Group task: Identify the actors in the JIT-bus app?

Use Case Identification

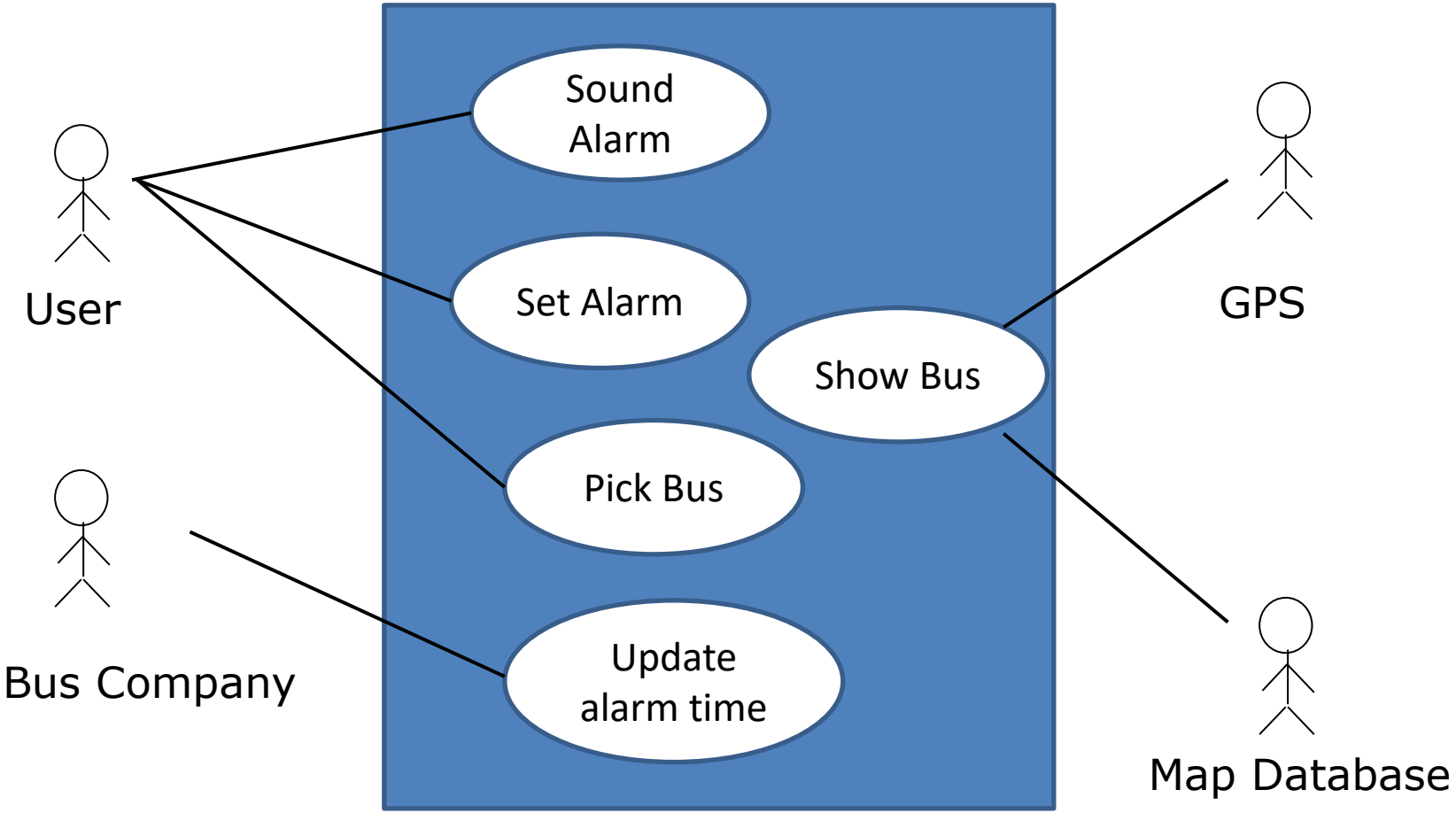
UWE Bristol University of the West of England

- Identifying Use Cases: go through all the actors of your system and ask yourself questions like:
  - What functions will the actor want?
  - What information is stored in the system? What actors will create, read, edit or delete this?
  - Does the system need to notify the actor of internal changes?

Draw a Use Case Diagram

Group task: Draw the use-case diagram for the JIT-bus app

Use Cases diagram for JIT-bus



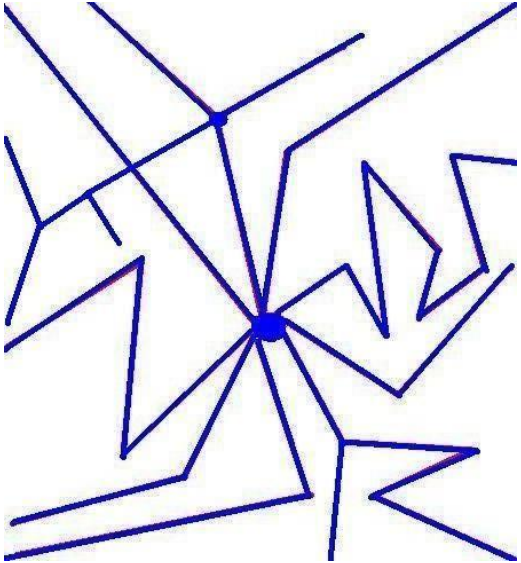
Documenting Use Cases

UWE Bristol University of the West of England

- Base case
- May include flow control (base case shouldn't)
- Must have pre and post conditions
- May be formatted to suit your style/need

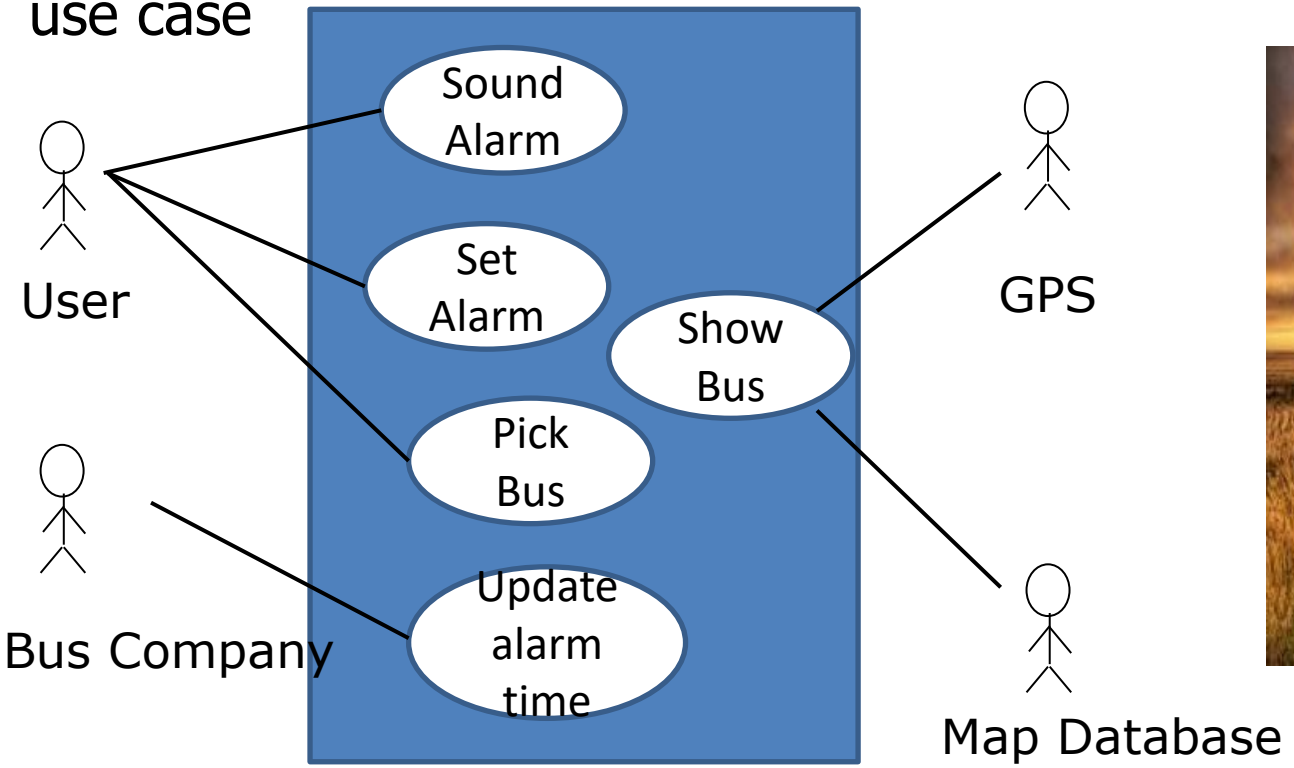
Paths

- Base path – ‘happy day’ scenario
- Alternative paths: Every other possible way the system can be (ab)used. Includes perfectly normal alternative use, but also errors and failures.



Exercise: Write Base Path

- Group task: Write the base path for the 'set alarm' use case



Answer: Base Path

Precondition: A user has the app on a smart phone with network
Flow of events:
Basic path
1. The Use Case begins when the user opens the app
2. The map of the user's nearest bus stops is loaded
3. The user selects a bus stop and bus line
4. The alarm is set
5. Time elapses, alarm is modified when busses are off-schedule
6. Alarm goes off
7. User turns off alarm and the Use Case ends

Postcondition: The bus has left the selected bus stop

Alternative Paths

- How to find them?
  - Going line by line through the Base Path
  - By exception category:
    - Actor exits application
    - Actor cancels operation
    - Actor provides incomplete/bad data
    - System crashes
    - …
- Work out significant alternate paths:
  1. In base description
  2. Under base description as separate paragraphs
  3. In entirely separate descriptions

Exercise: Write Alternate Paths

- Explore what alternate paths there may be for the ‘set alarm’ use case

- Document the most significant alternate path as a paragraph below the base path

Answer: Alternate Paths

1. User cancels alarm
2. User changes bus
3. User moves
4. GPS signal lost
5. Connection to bus service lost
6. Connection to internet lost
7. User exits application
8. …

Answer: most significant alternative path

UWE Bristol University of the West of England

- Flow of events:
- *Basic path*

1. The Use Case begins when the user opens the app
2. The map of the user's nearest bus stops is loaded
3. The user selects a bus stop and bus line
4. The alarm is set
5. Time elapses, alarm is modified when busses are off-schedule
6. Alarm goes off
7. User turns off alarm and the Use Case ends

- *Alternative paths:*

If at any time during step 5 the user selects a different bus line, the current alarm is disabled, and the basic path continues from step 4

OOA Overview – Detailed

UWE Bristol University of the West of England

Idea

- One-paragraph outline of idea
- No details whatsoever

Project Description

- Brief but complete
- Small group of authors
- What you want to accomplish, not how

Boundary Analysis

- What's inside -> create
- What's outside -> interface

OOA Overview – Detailed cont.

UWE Bristol University of the West of England

Use Case Identification

- (almost) Always initiated by Actors
- Short-term

Use Case Diagrams

- Various ways of displaying
- Includes context diagrams, activity diagrams

Path analysis

- Find Base Path
- Find Alternative Paths

Use Case Documentation

- Use style that suits you and audience best
- Include Base and Significant Alternative Paths

Suggested reading

UWE Bristol University of the West of England

- Suggested reading for this lecture
  - Object-oriented software engineering : using UML, patterns, and Java / Bernd Bruegge & Allen H. Dutoit, Chapters 1-2
  - Applying Use Cases, a practical guide (2nd ed.), by Schneider&Winters, Chapters 1-4
  - UML Distilled, by Martin Fowler & Kendal Scott, Chapters 1-3

- Acknowledgement: some slides adapted from PJ Davies, UBC, for ECE 314, Real-time Software Engineering

- **Next Lecture**
  - How to model the static (logical) structure of a system using Class Diagrams
  - How to model the way objects interact using Sequence Diagrams
  - How to model the object's decision what to do when receiving a message using State Machine Diagrams and Activity Diagrams